

VeloGrad: Local Value-Gradient Learning for Reinforcement Fine-Tuning of Flow Policies

Anonymous Authors

Abstract—Flow policies generate actions through a sequence of velocity updates, so the value of a local velocity choice depends on the remaining generation process. Capturing this dependence is important for guiding policy improvement with action values. We propose VeloGrad, which learns a velocity-conditioned continuation critic to provide a local derivative signal for reinforcement learning. The critic predicts the value of the action produced by taking one step with a candidate velocity and completing generation with the current policy. To directly train the derivatives used by the actor, we complement value distillation with central-difference supervision from symmetric finite-radius velocity perturbations. This enables actor optimization without backpropagating through the flow solver, while deployment retains standard sampling without critic queries. Experiments cover offline and offline-to-online learning on OGBench and Robomimic, together with real-robot adaptation to target disturbances and multi-stage manipulation. VeloGrad raises average offline success across four OGBench domains from 66.5% with QAM to 79.6%. Over two independent real-robot training runs, it reaches 98.3% mean success on disturbed flower insertion and improves autonomous end-to-end windmill activation from 53.3% with QAM to 71.7%.

Index Terms—Flow matching, reinforcement learning, robot manipulation, policy fine-tuning, value gradient

I. INTRODUCTION

Diffusion and flow policies learn complex, multimodal robot behaviors from demonstrations [1], [2]. Yet these policies can fail when targets move during execution, dynamic interactions require precise control, or errors accumulate across manipulation stages. Reward feedback from these executions offers a way to improve behavior without collecting new demonstrations for every failure. We study reinforcement learning (RL) for fine-tuning pretrained flow policies in this setting.

Value-based RL improves a policy using an action critic that estimates the return associated with its output. For a single-step actor, the action-value gradient can be backpropagated directly through the predicted action. A flow policy instead constructs an action through a sequence of velocity predictions. Changing one velocity changes the input to all subsequent flow steps, so its effect on action value depends on how the remaining generation process transforms that change. Backpropagating through the solver captures this dependence, but introduces computational cost and sensitivity to the conditioning of the flow dynamics [3], [4].

Existing approaches address this optimization problem through auxiliary actors or latent steering [3], [5], redesigned samplers [6], or local training objectives for the flow field [4], [7], [8]. We investigate learning a value function conditioned on the local velocity decision. This function can represent the

effect of the remaining generation process while providing an actor-update signal through a single velocity prediction.

We introduce VeloGrad, which learns this function with a velocity-conditioned continuation critic. At a replay-interpolated flow state, we apply a candidate velocity for one integration step and complete generation with the current policy held fixed. The action critic scores the resulting action to supervise the continuation critic. Because scalar value fitting does not directly supervise the gradient used by the actor, we also perturb the velocity in opposite directions and use the normalized difference between the resulting action values as a directional target. The actor updates through the learned critic’s gradient with respect to velocity, without backpropagating through the remaining solver. Deployment uses the standard flow sampler without critic queries.

Our contributions are:

- 1) **Velocity-conditioned continuation values.** We learn the downstream action value of a candidate flow velocity, providing a local actor-update signal that incorporates the remaining generation dynamics through supervision.
- 2) **Finite-radius derivative supervision.** We supervise velocity derivatives with symmetric downstream evaluations and show that the expected sensitivity loss targets the gradient of a ball-smoothed continuation value.
- 3) **Simulation and real-robot evaluation.** We evaluate offline and offline-to-online learning on OGBench and Robomimic, and test target disturbance, elastic launching, and long-horizon multi-stage manipulation on a Piper-X robot.

II. RELATED WORK

Real-world reinforcement fine-tuning of robot policies. Off-policy RL reuses experience and demonstrations for data-efficient learning [9]. Robotic systems combine visual perception [10] with human corrections for contact-rich and dynamic manipulation [11]. Recent work adapts demonstration-trained generative policies [5], [12], [13] and vision-language-action models [14]–[16] through RL. Broader post-training uses iterative policy refinement [17] or specialist-to-generalist distillation [18].

Reinforcement learning with diffusion and flow policies. Offline methods combine generative behavior modeling with value optimization or behavior constraints [19], [20]; policy-gradient methods optimize denoising or flow-matching objectives [21]–[23]. Other approaches separate behavior modeling from value maximization through one-step actors [3], [24],

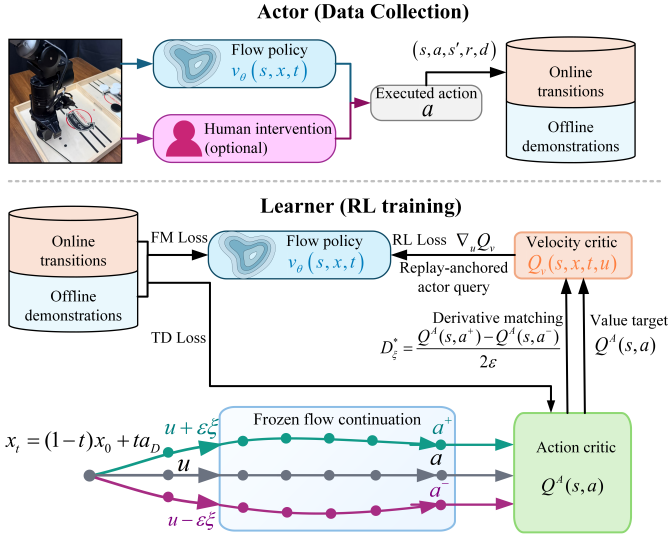


Fig. 1. Overview of VeloGrad. Top: the actor collects transitions with optional human intervention. Bottom: the learner trains a velocity critic on frozen flow flow continuations and updates the flow policy with the critic derivative $\nabla_u Q_v$. Network parameter subscripts are omitted.

learn values over action chunks [25], or adapt policies through residual, edit-based, or latent interfaces [5], [12], [26]–[28].

Value-gradient methods [29] extend critic-based improvement to score supervision [30] and differentiable latent mappings [31]. QAM [4] uses lean adjoint dynamics under a learned behavior policy to propagate terminal action-critic gradients into velocity supervision. FlowDPG [7] constructs velocity targets by combining one-step clean-action projections with scaled critic-gradient corrections. Q-VGM [8] refines one-step clean-action estimates through critic-gradient ascent and matches the resulting velocity corrections with the residual flow field.

VINE designs the sampler for stable end-to-end RL [6], while QGF guides flow sampling with action-critic gradients at test time [32]. Accurate proxy values do not necessarily provide useful policy gradients [31]; derivative supervision has also been studied in Sobolev training [33]. VeloGrad learns a scalar value conditioned on a candidate velocity. Forward continuations and scalar action-value evaluations supervise both this value and its velocity derivative, which provides the actor-update signal.

III. METHOD

VeloGrad combines an action critic that evaluates complete action chunks, a velocity critic trained on continuation values and directional targets, and a flow actor updated through the velocity critic. Velocity-critic supervision and actor updates use the same distribution of replay-interpolated queries (Fig. 1).

A. Flow Policies and Action Values

Let s denote the environment observation supplied to the policy and $a \in \mathbb{R}^d$ an H -step action chunk, with per-step

dimension d_a and $d = Hd_a$. The flow state x evolves over flow time $t \in [0, 1]$. Starting from $x_0 \sim \mathcal{N}(0, I)$, the flow policy $v_\theta(s, x, t)$ generates a using K Euler steps:

$$x_{k+1} = x_k + \Delta t v_\theta(s, x_k, t_k), \quad \Delta t = 1/K, \quad t_k = k/K. \quad (1)$$

Intermediate states are clipped after each step, and the terminal output $a = x_K$ is clipped to the action bounds. Here, terminal refers to the end of flow generation at $t = 1$. Clipping is omitted from (1) for readability.

For flow matching, we sample a replay pair $(s, a_D) \sim \mathcal{D}$, noise $x_0 \sim \mathcal{N}(0, I)$, and time $t \sim \text{Unif}(0, 1)$, and construct

$$x_t^{\mathcal{D}} = (1-t)x_0 + ta_D. \quad (2)$$

The loss $\mathcal{L}_{\text{FM}}$ regresses $v_\theta(s, x_t^{\mathcal{D}}, t)$ toward $a_D - x_0$ by mean-squared error. The replay buffer initially contains offline data and incorporates new experience during online adaptation. We denote the induced distribution of $(s, x_t^{\mathcal{D}}, t)$ by $\rho_{\mathcal{D}}$ and use it for both velocity-critic supervision and actor updates.

Following Q-Chunking [25], an action critic $Q_\phi^A(s, a)$ evaluates the complete chunk. For a replay segment with rewards $r_0, \dots, r_{H-1}$ and next observation s' after the chunk, we sample $a' \sim \pi_\theta(\cdot | s')$ and minimize

$$y_{\text{TD}} = \sum_{j=0}^{H-1} \gamma^j r_j + \gamma^H m \bar{Q}_{\phi^-}^A(s', a'), \quad (3)$$

$$\mathcal{L}_{Q^A} = \mathbb{E} \left[(Q_\phi^A(s, a) - \text{sg}(y_{\text{TD}}))^2 \right].$$

Here, π_θ is induced by (1), γ is the discount factor, ϕ^- denotes target-critic parameters, and sg stops gradients. The terminal mask m controls bootstrapping. An overbar on Q denotes the mean of two critic heads; $\bar{Q}_{\phi^-}^A$ uses target heads. Rewards use the corresponding termination masks; padded steps are masked in chunk losses. We omit head indices in per-head expressions.

B. Local Continuation Values

At a replay-interpolated query $(s, x, t) \sim \rho_{\mathcal{D}}$, we evaluate a candidate velocity u by applying it for one integration step and completing generation with the current policy held fixed:

$$Q_{\theta_{\text{ref}}}(s, x, t, u) = Q_\phi^A \left(s, \Phi_{\theta_{\text{ref}}}^{t+h(t) \rightarrow 1}(x + h(t)u) \right), \quad (4)$$

$$h(t) = \min(1/K, 1-t), \quad \theta_{\text{ref}} = \text{sg}(\theta).$$

The operator $\Phi_{\theta_{\text{ref}}}^{\tau \rightarrow 1}$ completes generation under the frozen reference policy from the actual time $\tau = t + h(t)$, without rounding to the original Euler grid. It uses step size $1/K$, shortening only the last step to end exactly at $t = 1$. The input, intermediate states, and terminal action use the sampler's clipping, including when $\tau = 1$.

The replay interpolant anchors the query to the data action. The observation s remains fixed during continuation, and Q^A evaluates the generated endpoint. Targets require only forward flow rollouts and action-critic evaluations, with no additional environment interactions or solver differentiation during actor updates.

Let $y = x + h(t)u$ and $a = \Phi_{\theta_{\text{ref}}}^{t+h(t) \rightarrow 1}(y)$. At points where the continuation and action-critic derivatives exist, the chain rule gives

$$\nabla_u Q_{\theta_{\text{ref}}} = h(t) \left(\frac{\partial a}{\partial y} \right)^\top \nabla_a Q_\phi^A(s, a). \quad (5)$$

This derivative includes the sensitivity of the remaining flow dynamics and clipping operations. The continuation value $Q_{\theta_{\text{ref}}}$ is computed from the action critic and frozen flow. We learn a velocity critic $Q_{v,\psi}(s, x, t, u)$ to approximate its value and velocity derivative and supply a local actor-update signal.

C. Value and Directional Supervision

We draw $(s, x, t) \sim \rho_{\mathcal{D}}$ and set $u = v_{\theta_{\text{ref}}}(s, x, t)$. Applying (4) gives the center endpoint a_c , whose action value anchors the velocity critic:

$$\ell_{\text{center}} = [Q_{v,\psi}(s, x, t, u) - \text{sg}(Q_\phi^A(s, a_c))]^2. \quad (6)$$

Value regression alone does not directly supervise the gradient with respect to velocity. We sample a unit direction ξ uniformly from $\mathbb{S}^{d-1}$ and perturb the velocity by a fixed radius $\epsilon > 0$, giving $u^\pm = u \pm \epsilon\xi$. Continuing from the same (s, x, t) with these velocities gives endpoints $a^\pm$ and the directional target

$$D_\xi^* = \frac{Q_\phi^A(s, a^+) - Q_\phi^A(s, a^-)}{2\epsilon}. \quad (7)$$

We fit the critic’s directional derivative to this target:

$$\ell_{\text{sens}} = [\xi^\top \nabla_u Q_{v,\psi}(s, x, t, u) - \text{sg}(D_\xi^*)]^2, \quad (8)$$

yielding the training objective

$$\mathcal{L}_{Q_v} = \mathbb{E}_{(s,x,t) \sim \rho_{\mathcal{D}}, \xi} [\ell_{\text{center}} + \beta \ell_{\text{sens}}]. \quad (9)$$

The actor and action critic are fixed while constructing all three continuation targets. Only the center value is regressed; the perturbed endpoints provide derivative targets. Optimizing (8) differentiates the velocity critic’s directional derivative with respect to ψ , without backpropagating through the continuations.

Finite-radius interpretation. At a finite perturbation radius, the expected sensitivity loss targets the gradient of a ball-smoothed continuation value. We establish this relationship for a fixed reference actor, action critic, and replay distribution. For a query $z = (s, x, t) \sim \rho_{\mathcal{D}}$, write $f_z(w) = Q_{\theta_{\text{ref}}}(s, x, t, w)$, $u_z = v_{\theta_{\text{ref}}}(s, x, t)$, and $g_z = \nabla_u Q_{v,\psi}(s, x, t, u_z)$. Define the ball-smoothed continuation value $f_{z,\epsilon}(u) = \mathbb{E}_{b \sim \text{Unif}(\mathbb{B}^d)} [f_z(u + \epsilon b)]$, where $\mathbb{B}^d$ is the unit ball. If f_z is locally Lipschitz in velocity and the relevant second moments are finite,

$$\begin{aligned} \nabla_u f_{z,\epsilon}(u_z) &= d \mathbb{E}_\xi [\xi D_\xi^*], \\ \mathbb{E}_{z,\xi} [\ell_{\text{sens}}] &= \frac{1}{d} \mathbb{E}_z \|g_z - \nabla_u f_{z,\epsilon}(u_z)\|_2^2 + C, \end{aligned} \quad (10)$$

where C is independent of ψ for the fixed teacher. Coordinatewise clipping is 1-Lipschitz and preserves local Lipschitz continuity under composition. The first identity follows from

Algorithm 1 VeloGrad: Joint Update with Replay-Anchored Supervision

Require: Replay buffer $\mathcal{D}$, actor v_θ , critics $Q_\phi^A, Q_{v,\psi}$

- 1: Initialize target parameters $\phi^- \leftarrow \phi$
 - 2: **for** each training iteration **do**
 - 3: Sample a replay minibatch $(s, a_D, \{r_j\}, s', m)$
 - 4: Sample $a' \sim \pi_\theta(\cdot | s')$ and form $\mathcal{L}_{Q^A}$ via (3)
 - 5: Fix the current reference actor $\theta_{\text{ref}} \leftarrow \text{sg}(\theta)$
 - 6: Draw $x_0 \sim \mathcal{N}(0, I)$ and $t \sim \text{Unif}(0, 1)$ per sample
 - 7: $x \leftarrow (1-t)x_0 + ta_D$; $u \leftarrow v_{\theta_{\text{ref}}}(s, x, t)$
 - 8: Draw $\xi \sim \text{Unif}(\mathbb{S}^{d-1})$; set $u^\pm \leftarrow u \pm \epsilon\xi$
 - 9: Continue from (s, x, t) using u, u^+, u^- to obtain a_c, a^+, a^- via (4)
 - 10: Form fixed targets with Q_ϕ^A and compute $\mathcal{L}_{Q_v}$ via (6)–(9)
 - 11: **if** an actor update is scheduled **then**
 - 12: Draw fresh noise and time; construct a replay interpolant and compute $\mathcal{L}_{\text{actor}}$ via (11)
 - 13: **end if**
 - 14: Update ϕ, ψ with their losses and θ when scheduled
 - 15: Update the target: $\phi^- \leftarrow (1 - \tau_Q)\phi^- + \tau_Q\phi$
 - 16: **end for**
-

ball smoothing [34] and spherical symmetry; the second uses $\mathbb{E}_\xi [\xi \xi^\top] = I/d$. The center target remains the unsmoothed value.

D. Local Actor Updates

Using fresh noise and time draws with the replay minibatch, we update the actor on the same interpolation distribution:

$$\begin{aligned} \mathcal{L}_{\text{actor}} &= \lambda_{\text{FM}} \mathcal{L}_{\text{FM}} \\ &\quad - \lambda_Q \mathbb{E}_{(s,x,t) \sim \rho_{\mathcal{D}}} [\bar{Q}_{v,\psi}(s, x, t, v_\theta(s, x, t))]. \end{aligned} \quad (11)$$

Here $\bar{Q}_{v,\psi}$ is the mean velocity-critic value. With the critic and sampled query fixed, the value term has parameter gradient

$$\nabla_\theta [-\bar{Q}_{v,\psi}] = - \left(\frac{\partial v_\theta}{\partial \theta} \right)^\top \nabla_u \bar{Q}_{v,\psi}. \quad (12)$$

The critic provides a local surrogate update under the reference continuation policy, without solver backpropagation. The FM and value terms share an interpolation sample; critic supervision uses independent noise and time draws from the same distribution. Deployment uses Gaussian noise and the standard sampler in (1), without critic queries.

Implementation. Both critics have two heads with mean aggregation. Each Q_v head receives fixed value and directional-derivative targets from the corresponding pre-update Q^A head. We use one spherical direction per query, $K = 10$, $\epsilon = 0.1$, $\beta = 1$, $\lambda_Q = 1$, and $\tau_Q = 0.005$. The reference actor is refreshed from the current actor as learning proceeds. Algorithm 1 shows the joint-update form; separately scheduled actor updates use the latest critic. Task-specific FM weights are specified below.

TABLE I
OGBENCH SUCCESS RATE (%; MEAN [95% BOOTSTRAP CI]). OFFLINE
USES 1M GRADIENT STEPS; O2O ADDS 1M ENVIRONMENT STEPS. P33:
PUZZLE-3 $\times$ 3; C2/C3: CUBE-DOUBLE/TRIPLE.

Setting	Method	Scene	P33	C2	C3	Avg.
Offline	DAC	[70.8, 73.0]	[59.2, 85.4]	[34.3, 41.6]	[0.8, 3.6]	[43.4, 48.1]
	DSRL	99.9	85.0	75.5	1.6	65.5
	QAM	[97.0, 98.4]	[100.0, 100.0]	[62.2, 66.6]	[3.5, 5.5]	[66.0, 67.2]
	QC-FQL	[83.2, 83.8]	[38.8, 69.4]	[19.6, 43.7]	[2.5, 5.6]	[41.5, 50.3]
	FlowDPG	[95.8, 97.1]	[93.5, 97.8]	[65.9, 72.9]	[8.6, 14.8]	[67.0, 68.7]
	VINE [†]	[73, 73]	[95, 95]	[4, 4]	[1, 1]	[43, 43]
	VeloGrad	[97.2, 98.6]	100.0	75.8	44.7	79.6
O2O	DAC	[96.0, 98.9]	[30.0, 46.0]	[99.4, 100.0]	[30.6, 40.4]	[65.3, 69.8]
	DSRL	100.0	84.8	99.9	0.6	71.3
	QAM	[99.1, 100.0]	[100.0, 100.0]	[99.7, 100.0]	[78.7, 79.9]	[94.6, 94.9]
	QC-FQL	[99.0, 99.6]	[100.0, 100.0]	[99.1, 100.0]	[56.3, 61.0]	[88.6, 90.1]
	FlowDPG	[99.0, 100.0]	[100.0, 100.0]	[99.7, 100.0]	[70.4, 84.6]	[92.6, 95.9]
	RLPD	[91.4, 94.0]	[100.0, 100.0]	[97.8, 99.6]	[33.8, 46.2]	[81.5, 84.4]
	VeloGrad	[99.5, 100.0]	100.0	99.6	93.4	98.2

IV. EXPERIMENTS

A. Simulation Experiments

Benchmarks and evaluation. We evaluate four OGBench manipulation domains [35]: Scene, Puzzle-3 $\times$ 3, Cube-Double, and Cube-Triple, using $H = 5$ action chunks and the default `-play` datasets. The Cube domains use dense rewards; Scene and Puzzle use sparse rewards. We also evaluate Lift, Can, Square, and Transport from the Robomimic multi-human datasets with low-dimensional observations [36], using $H = 5$ for Lift/Can/Square and $H = 8$ for Transport.

For both benchmarks, offline training uses 1M gradient steps and offline-to-online (O2O) adaptation adds 1M environment steps. Methods adapt from their offline checkpoints under a common evaluation protocol while retaining their own update, replay, and optimization strategies; RLPD is evaluated only online. We report final-checkpoint success over 12 training seeds for OGBench and four for Robomimic. For OGBench, scores are averaged over the five tasks within each domain and seed; means and 95% bootstrap confidence intervals are then computed across seeds.

Baselines and hyperparameters. We reproduce baselines with the released QAM codebase and follow its evaluation protocol [4]. QAM, QC-FQL, DAC, and DSRL use the best-performing configurations provided in the released QAM implementation [4]. For FlowDPG, we conduct our own hyperparameter searches and report results using the best-performing configurations among those evaluated. For VeloGrad, we select λ_{FM} from $\{10, 25, 50, 75, 100\}$ on OGBench and $\{100, 250, 500, 750, 1000\}$ on Robomimic using preliminary seeds. Selection is based on success rate together with inspection of critic estimates for overestimation, after which the configuration is fixed for the reported evaluation. The

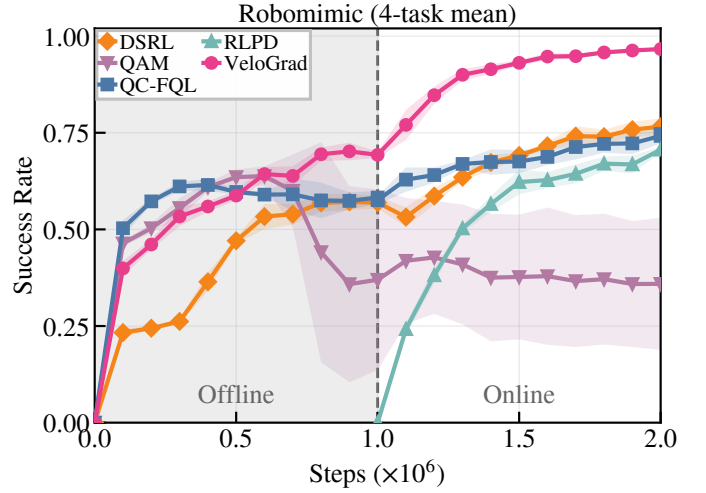


Fig. 2. Robomimic multi-human success with low-dimensional observations, averaged over four tasks within each seed. Curves show means and shading shows sample standard deviations across training seeds. Gray marks offline training; the dashed line starts online adaptation. The x-axis counts offline gradient updates, then online environment steps offset by 10^6 . RLPD is evaluated only online.

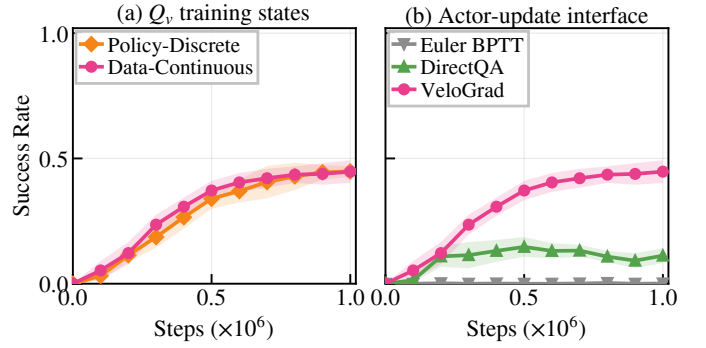


Fig. 3. Actor-update controls on offline Cube-Triple. Shaded bands are the sample standard deviation after averaging the five tasks within each seed. (a) Q_v trained on Policy-Discrete or Data-Continuous states, with actor queries fixed to Data-Continuous. (b) Euler backpropagation through time (BPTT) and DirectQA compared with VeloGrad’s learned velocity-value interface.

selected values are 10 by default on OGBench, 25 for Scene task 4 in both phases, and 50 for online Cube-Double/Triple; on Robomimic they are 100 for Lift/Can/Square and 750 for Transport.

VINE results ([†]) are taken from the original paper [6], which reports 12-seed results under the QAM evaluation protocol. Its Avg. is computed from the published domain means and omits a confidence interval because per-seed results are unavailable.

Main results. VeloGrad achieves the highest average success on OGBench: 79.6% offline and 98.2% after online adaptation, versus 66.5%/94.7% for QAM and 68.0%/94.0% for FlowDPG (Table I). On Cube-Triple, VeloGrad reaches 44.7%/93.4%, compared with 4.8%/79.4% for QAM and 11.7%/76.8% for FlowDPG. The largest gains over both baselines occur on Cube-Triple.

Across the four Robomimic tasks, VeloGrad reaches $96.7 \pm$

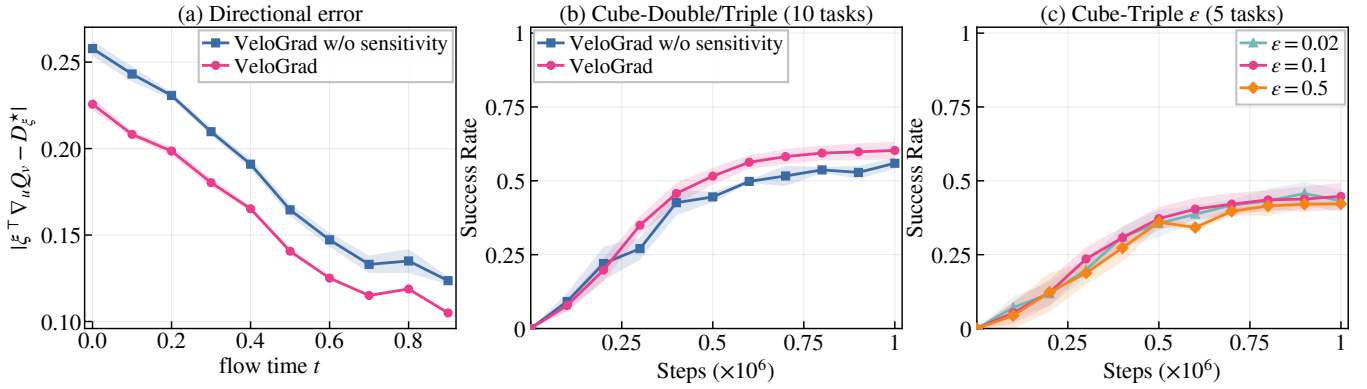


Fig. 4. Sensitivity controls with replay-interpolated Q_v supervision. VeloGrad w/o sensitivity regresses the center value; VeloGrad adds ℓ_{sens} from (8). Shaded bands are the sample standard deviation. (a) Directional error versus flow time: $|\xi^\top \nabla_u Q_v - D_\xi^*|$ averaged over 1,024 held-out Data-Continuous queries in Cube-Triple and four directions per query ($\epsilon = 0.1$), using each method’s own action-critic continuation targets. (b) Mean offline success on the ten Cube-Double and Cube-Triple tasks. (c) Mean success on the five Cube-Triple tasks for $\epsilon \in \{0.02, 0.1, 0.5\}$.

TABLE II
OFFLINE CUBE-TRIPLE SUCCESS FOR THE REPORTED CRITIC-TRAINING
AND ACTOR-QUERY COMBINATIONS.

Q_v training states	Actor queries	Success (%)
Policy-Discrete	Policy-Discrete	0.0
Policy-Discrete	Data-Discrete	15.1
Policy-Discrete	Data-Continuous	44.8
Data-Continuous	Data-Continuous	44.7

0.5% mean success after online adaptation, compared with $76.6 \pm 2.2\%$ for DSRL (Fig. 2).

Design choices. We separate critic-training states, actor-query locations, and actor-update interfaces on offline Cube-Triple. Data-Continuous and Data-Discrete denote replay interpolants at continuous times and Euler times t_k , respectively; Policy-Discrete denotes current-policy solver states x_k at t_k . Table II lists the reported combinations of critic-training states and actor queries.

These controls suggest that the actor-query distribution is the more consequential design choice in this setting. With Data-Continuous actor queries fixed, changing the critic-training states shows no clear advantage (44.8% versus 44.7%; Fig. 3(a)). With Policy-Discrete supervision fixed, Data-Continuous actor queries perform best among the reported query conditions (Table II).

Finally, Fig. 3(b) compares actor-update interfaces. DirectQA maximizes $\bar{Q}^A(s, \hat{a})$ through $\hat{a} = \text{clip}(x_t^D + (1 - t)u, -1, 1)$, with $u = v_\theta(s, x_t^D, t)$, and reaches $11.2 \pm 3.0\%$ (mean $\pm$ sample standard deviation), compared with 44.7% for VeloGrad in Table I. Euler backpropagation through time (BPTT) remains near zero with the standard sampler used here; this control does not include the sampler redesign introduced by VINE [6].

Separately, sampling FM and RL query times independently yields $46.0 \pm 3.3\%$ success (mean $\pm$ sample standard deviation), indicating that shared time draws are not required.

TABLE III
PER-STEP TRAINING TIME ON CUBE-TRIPLE.

Method	Time (ms/step)
QC-FQL	5.228
QAM	7.034
VeloGrad w/o sensitivity	6.065
VeloGrad	7.048

Directional sensitivity supervision. Adding sensitivity supervision reduces directional error by approximately 16% (Fig. 4(a)). The methods share replay samples, noise, query times, and test directions, but use their own action critics and continuation policies to construct targets; this diagnostic therefore measures agreement with learned continuation targets.

On the ten Cube-Double and Cube-Triple tasks, offline success rises from 56.0% with VeloGrad w/o sensitivity to 60.3% with VeloGrad (Fig. 4(b)). Mean offline success across the four Robomimic tasks also rises from 65.8% to 69.3%.

Final success remains similar across $\epsilon \in \{0.02, 0.1, 0.5\}$, suggesting robustness to the perturbation radius within the tested range (Fig. 4(c)). We use $\epsilon = 0.1$ by default.

Training cost. Training times are averaged over 10,000 steps on OGBench Cube-Triple task 1 using one NVIDIA RTX 4090D GPU, batch size 256, and action-chunk horizon $H = 5$ for all methods. VeloGrad has comparable per-step training cost to QAM; sensitivity supervision adds 16.2% overhead over VeloGrad w/o sensitivity (Table III).

B. Real-Robot Offline-to-Online RL

Platform and comparisons. We evaluate three tasks under four conditions on a Piper-X: static and disturbed Flower Insertion, Elastic Puck Launching, and end-to-end (E2E) Windmill Activation. We compare against the pretrained BC Init, human-corrective imitation learning with HG-Dagger [37], and DSRL and QAM [4], [5]. Online methods share demonstrations, tasks, rewards, trajectory budgets, control frequency, chunk horizons, and correction and evaluation protocols, while retaining their optimization and replay strategies.



Fig. 5. Piper-X tasks: Flower Insertion, Elastic Puck Launching, and Windmill Activation (columns), each shown at three representative stages.

Observations comprise wrist and third-person RGB images, end-effector pose, and gripper state. Images are captured at 720×480 pixels, cropped to fixed task-specific regions, and resized to 128×128 pixels. Actions comprise six-dimensional Cartesian end-effector displacements and a gripper command. Each chunk executes fully at 25 Hz before replanning, subject to Cartesian workspace and per-step action bounds.

Tasks and rewards. Fig. 5 illustrates the tasks; Table IV summarizes chunk horizons, episode limits, and rewards. Other transitions receive zero reward. Operator buttons mark success and failure under the task criteria.

Flower Insertion. The robot starts holding a flower and must insert its stem into a vase initialized within a $10\text{ cm} \times 10\text{ cm}$ table region. Training includes vase displacements. In Flower-Static evaluation, the vase remains stationary. In Flower-Disturbed, it is displaced during robot motion to a randomly selected position within the reachable workspace. The policy must re-localize the vase and correct its approach.

Elastic Puck Launching. A single puck is initialized at a random position within a designated pickup region. Success requires grasping the puck and using an elastic band to launch it through the central opening into the opposite region. Evaluation initial-state distributions are matched across methods. Unlike insertion, which can allow corrections during contact, elastic launching requires suitable conditions before release, after which correction opportunities are limited. Release must synchronize gripper opening and upward motion to keep the band from catching on the gripper and disrupting the launch.

Windmill Activation. The robot starts holding a USB plug and inserts it into a fixed socket to activate the fan (stage 1, S_1). It then pushes a sand tray, initialized within a $5\text{ cm} \times 5\text{ cm}$ region, directly in front of the fan (stage 2, S_2). Finally, it grasps a windmill, inserts it into the sand tray, and positions it to rotate under the airflow (stage 3, S_3). The windmill starts within another $5\text{ cm} \times 5\text{ cm}$ region, with planar orientation sampled from $[-20^\circ, 20^\circ]$.

TABLE IV
TASK-SPECIFIC REAL-ROBOT SETTINGS. STAGE REWARDS ARE GIVEN ONLY ON FIRST COMPLETION.

Task	H	Limit (steps)	Reward
Flower Insertion	16	400	Success: 10
Puck Launching	16	500	Success: 10 S_1/S_2 : 2 each
Windmill Activation	32	1000	Full task: 10

TABLE V
FINAL AUTONOMOUS SUCCESS OVER TWO TRAINING RUNS (30 TRIALS PER RUN). CELLS SHOW MEAN SUCCESS (%) AND SUCCESSFUL TRIAL COUNTS (RUN 1, RUN 2). AVG. IS THE UNWEIGHTED MEAN OVER THE FOUR CONDITIONS; BOLD MARKS THE BEST MEAN.

Method	Flower Static	Flower Disturbed	Puck	Windmill E2E	Avg.
BC Init	31.7 (8, 11)	8.3 (3, 2)	1.7 (0, 1)	0.0 (0, 0)	10.4
HG-Dagger	68.3 (21, 20)	68.3 (23, 18)	25.0 (9, 6)	28.3 (7, 10)	47.5
DSRL	70.0 (24, 18)	31.7 (7, 12)	28.3 (8, 9)	45.0 (16, 11)	43.8
QAM	90.0 (26, 28)	83.3 (23, 27)	28.3 (10, 7)	53.3 (15, 17)	63.8
VeloGrad	95.0 (29, 28)	98.3 (29, 30)	46.7 (13, 15)	71.7 (21, 22)	77.9

Implementation. Each task starts with 10 teleoperated demonstrations and 30,000 flow-matching updates; the demonstrations also initialize replay. Windmill S_1 uses an Xbox controller for teleoperation; all other demonstrations use a leader arm. During online learning, operator corrections replace policy actions and enter replay for off-policy updates.

VeloGrad uses separate per-camera ResNet-10 backbones [38] for the actor and critics, with the same ImageNet initialization and spatial-softmax heads [39]. We concatenate proprioception with 64 features per camera. The actor is a conditional 1D U-Net with widths $[128, 256, 512]$. Q^A and Q_v share the critic encoder and each use two three-layer, 512-unit MLP heads. Actor backbones use $0.1 \times$ the actor learning rate; critic backbones are frozen, with trainable spatial-softmax heads.

For VeloGrad, online transitions and demonstrations share one replay buffer, without a separate offline/online sampling ratio. The actor uses learning rate 3×10^{-4} and batch size 256 during both pretraining and online learning; online critics also use 3×10^{-4} . We use $\lambda_{\text{FM}} = 100$ and $\gamma = 0.98$. Each learner iteration performs four critic-only updates followed by one joint actor-critic update, training both Q^A and Q_v in every critic update. The learner performs approximately one iteration per environment step for Flower Insertion and Elastic Puck Launching and three for Windmill Activation, publishing parameters every 200 critic updates.

Evaluation protocol. For each online method and task, we conduct two independent training runs, each with 100 training trajectories. Checkpoints after 25, 50, 75, and 100 trajectories are evaluated using 30 autonomous trials per run and condition, with manual resets between trials. Autonomous success requires full-task completion within the episode limit, without corrective teleoperation or manual recovery; Windmill E2E success requires the entire three-stage sequence. Table V reports final-checkpoint means, and Fig. 6 shows checkpoint

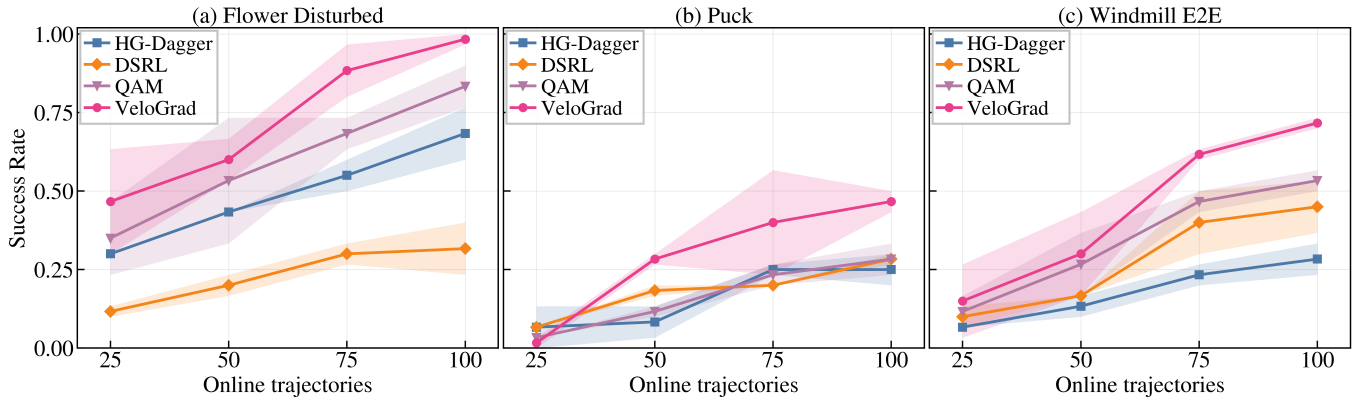


Fig. 6. Autonomous success on (a) Flower Disturbed, (b) Puck, and (c) Windmill E2E. Each checkpoint uses 30 trials per run after 25, 50, 75, and 100 online trajectories. Lines show means over two independent training runs; shading shows run-to-run ranges, not confidence intervals.

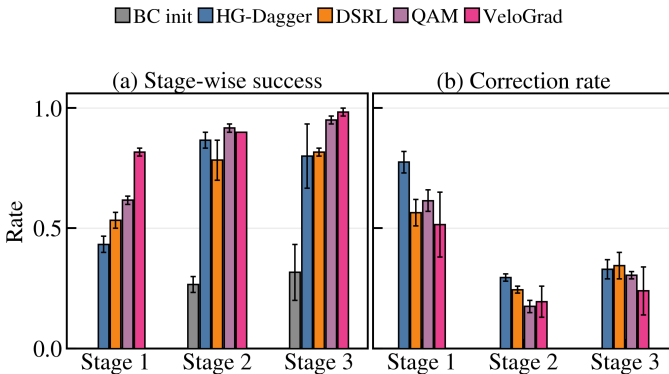


Fig. 7. Windmill diagnostics over two training runs. (a) Stage-wise success on 30 final-checkpoint trials per run. Resets after failures serve only stage diagnostics; the E2E trial remains unsuccessful. (b) Fraction of 100 online trajectories per run with human correction. Bars show two-run means; whiskers show ranges. BC Init appears only in (a).

progress.

Windmill stage rates each use all 30 final-checkpoint trials per run. After a failure, the operator restores the scene for the next stage; the failed stage and E2E trial remain unsuccessful. Fig. 7 reports these stage diagnostics and the fraction of online trajectories with human correction.

Results. Final autonomous performance. VeloGrad achieves the highest mean autonomous success in all four conditions (Table V). The largest gains over QAM are on Puck and Windmill E2E: 46.7% versus 28.3% and 71.7% versus 53.3%, respectively. It matches or exceeds QAM in all eight condition-run comparisons, with higher success rates in seven.

Online adaptation. VeloGrad leads the two-run means at 50, 75, and 100 trajectories on Flower-Disturbed, and at the later checkpoints on Windmill E2E and Puck (Fig. 6).

Stage-wise performance. VeloGrad leads mean success in both insertion stages, while QAM is slightly higher on sand-tray positioning (Fig. 7(a)). It has the lowest mean Stage 1 correction rate and a lower final-stage correction rate than QAM (Fig. 7(b)).

C. Real-Robot Offline Ablation

We pool all 200 USB insertion episodes from the two VeloGrad online-training runs on Windmill Activation, including human interventions. The 172 successful episodes (86.0%) average 194.4 steps. BC, VeloGrad w/o sensitivity, and VeloGrad each train for 50,000 actor updates on this fixed dataset without new training interactions, followed by autonomous final-checkpoint evaluation (Table VI). As in Fig. 4, VeloGrad w/o sensitivity retains continuation-value regression but omits ℓ_{sens} .

TABLE VI
AUTONOMOUS USB INSERTION AFTER 50,000 OFFLINE ACTOR UPDATES.
EPISODE LENGTHS ARE AVERAGED OVER SUCCESSFUL TRIALS.

Method	Success (%) $\uparrow$	Length (steps) $\downarrow$
BC	43.3	295.3
VeloGrad w/o sensitivity	75.0	232.1
VeloGrad	81.7	219.7

VeloGrad achieves 81.7% success versus 75.0% for VeloGrad w/o sensitivity and 43.3% for BC. Sensitivity supervision adds 6.7 percentage points and reduces mean successful-episode length from 232.1 to 219.7 steps.

V. LIMITATIONS

Scaling to larger vision-language-action or world-action models remains untested; multiple forward continuations may be costly at these scales. The velocity critic inherits action-critic errors, which can introduce noise into finite-difference targets. Larger action spaces may also require more than one supervision direction per query, further increasing target-construction cost. Real-robot evaluation is limited to three tasks on a single platform.

VI. CONCLUSION

We presented VeloGrad, which learns a velocity-conditioned continuation critic using value distillation and finite-radius derivative supervision. The critic provides a local actor-update signal that accounts for the remaining generation process without solver backpropagation, while deployment retains

standard flow sampling without critic queries. Experiments on OGBench, Robomimic, and a Piper-X robot demonstrate improved performance in offline and offline-to-online learning, including adaptation to target disturbances and multi-stage manipulation. Ablations further highlight the importance of replay-anchored actor queries and the benefit of directly supervising the velocity derivatives used for policy optimization.

REFERENCES

- [1] C. Chi *et al.*, “Diffusion policy: visuomotor policy learning via action diffusion,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.
- [2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.
- [3] S. Park, Q. Li, and S. Levine, “Flow Q-learning,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 48104–48127.
- [4] Q. Li and S. Levine, “Q-learning with adjoint matching,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [5] A. Wagenmaker *et al.*, “Steering your diffusion policy with latent space reinforcement learning,” in *Proc. Conf. Robot Learn. (CoRL)*, 2025, pp. 258–282.
- [6] R. Yang *et al.*, “VINE: taming generative control policies for reinforcement learning,” *arXiv preprint arXiv:2607.10369*, 2026.
- [7] K. Shi *et al.*, “FlowDPG: deterministic policy gradient on flow matching policies for real-world manipulation,” *arXiv preprint arXiv:2606.22303*, 2026.
- [8] Z. Wang, Y. Liu, X. Mao, M. Wang, and Y. Mu, “Q-VGM: Q-value-gradient matching for offline-to-online reinforcement learning of flow-matching VLA,” *arXiv preprint arXiv:2606.08015*, 2026.
- [9] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 1577–1594.
- [10] J. Luo *et al.*, “SERL: a software suite for sample-efficient robotic reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [11] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *Science Robotics*, vol. 10, no. 105, Art. no. eads5033, 2025.
- [12] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, “Residual off-policy RL for finetuning behavior cloning policies,” *arXiv preprint arXiv:2509.19301*, 2025.
- [13] K. Lei *et al.*, “Performant robotic manipulation with real-world reinforcement learning,” *Science Robotics*, vol. 11, no. 116, Art. no. ead6267, 2026.
- [14] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: a reinforced fine-tuning method for VLA models via consistency policy,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [15] P. Dong, K.-H. Hung, T. Gao, D. Sadigh, and C. Finn, “EXPO-FT: sample-efficient reinforcement learning finetuning for vision-language-action models,” *arXiv preprint arXiv:2605.25477*, 2026.
- [16] C. Xu *et al.*, “RL Token: bootstrapping online RL with vision-language-action models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [17] Physical Intelligence *et al.*, “ $\pi_{0.6}^*$: a VLA that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [18] W. Xiao *et al.*, “Self-improving vision-language-action models with data generation via residual RL,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [19] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion policies as an expressive policy class for offline reinforcement learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.
- [20] L. Fang, R. Liu, J. Zhang, W. Wang, and B.-Y. Jing, “Diffusion actor-critic: formulating constrained policy iteration as diffusion noise regression for offline reinforcement learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [21] A. Z. Ren *et al.*, “Diffusion policy policy optimization,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [22] D. McAllister *et al.*, “Flow matching policy gradients,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [23] T. Zhang, C. Yu, S. Su, and Y. Wang, “ReinFlow: fine-tuning flow matching policy with online reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [24] F. Nguimatsia Tiofack, T. Le Hellard, F. Schramm, N. Perrin-Gilbert, and J. Carpentier, “Guided flow policy: learning from high-value actions in offline reinforcement learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [25] Q. Li, Z. Zhou, and S. Levine, “Reinforcement learning with action chunking,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [26] Z. Sun and S. Song, “From prior to pro: efficient skill mastery via distribution contractive RL finetuning,” *arXiv preprint arXiv:2603.10263*, 2026.
- [27] P. Dong, Q. Li, D. Sadigh, and C. Finn, “EXPO: stable reinforcement learning with expressive policies,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [28] D. Yu, K. Lei, Z. Jiang, J. Pan, and H. Xu, “Beyond action residuals: real-world robot policy steering via bottleneck latent reinforcement learning,” *arXiv preprint arXiv:2605.19919*, 2026.
- [29] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2014, pp. 387–395.
- [30] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a diffusion model policy from rewards via Q-score matching,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 41163–41182.
- [31] H. Im, A. Kolobov, J. Fu, and Y. Lee, “Latent policy steering through one-step flow policies,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2026.
- [32] Z. Zhou *et al.*, “Test-time gradient guidance of flow policies in reinforcement learning,” *arXiv preprint arXiv:2606.11087*, 2026.
- [33] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, “Sobolev training for neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [34] A. D. Flaxman, A. T. Kalai, and H. B. McMahan, “Online convex optimization in the bandit setting: gradient descent without a gradient,” in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2005, pp. 385–394.
- [35] S. Park, K. Frans, B. Eysenbach, and S. Levine, “OGBench: benchmarking offline goal-conditioned RL,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [36] A. Mandlekar *et al.*, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 1678–1690.
- [37] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “HG-Dagger: interactive imitation learning with human experts,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 8077–8083.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [39] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, 2016.